ThePnLab

Algorithmic Paper-Trading Platform

Technical Architecture & Methodology Report

thepnlab.com

github.com/Hamza-67/ThePnLab

Hamza Houir Alami

ENSAM Lille (Arts et Métiers ParisTech)

2025–2026 · ongoing

Dual Architecture	**Gemini 2.0** Flash LLM	**36+** Assets	**ML** Learning Layer	**37** Live Users

Paper trading only — no real capital involved.

Source code: thepnlab.com

Contents

1 Executive Summary **2**

2 System Architecture **2**
2.1 Dual-Application Design . . . 2
2.2 Full Technology Stack . . . 3
2.3 Repository Structure . . . 3

3 Educational Agent — `tradinglab/agent.py` **3**
3.1 Decision Logic . . . 4
3.2 Deliberate Error Injection . . . 4
3.3 Realism Parameters . . . 5

4 Production Screener — `bot_screener.py` **5**
4.1 Asset Universe (36 tickers, 9 categories) . . . 5
4.2 Parallel Architecture . . . 5
4.3 Technical Indicators . . . 6
4.4 Composite Score Function . . . 6

5 LLM Decision Engine — `bot_brain.py` **7**
5.1 Model Cascade . . . 7
5.2 ML Learning Layer . . . 7
5.3 Prompt Engineering — Five Context Blocks . . . 8
5.4 Hard Constraints Enforced on the LLM . . . 8
5.5 Structured JSON Output . . . 9

6 News & Macro Sentiment — `news.py` **9**
6.1 Trusted Source Filtering . . . 9
6.2 Macro Relevance Scoring . . . 10
6.3 Risk Score → Position Multiplier . . . 10

7 Risk Management Framework **10**
7.1 Four-Layer Position Sizing Pipeline . . . 10
7.2 Exit Strategy . . . 11

8 Connection to Operations Research **11**
8.1 Sequential Decision-Making — MDP Formalization . . . 11
8.2 Multi-Criteria Decision Making . . . 11
8.3 Stochastic Control & Regime Switching . . . 11

9 Results & Validation **12**
9.1 Live Performance — Last 30 Days . . . 12

10 Limitations & Research Roadmap **12**
10.1 Current Limitations . . . 12
10.2 Research Roadmap . . . 13

11 References **13**

1. Executive Summary

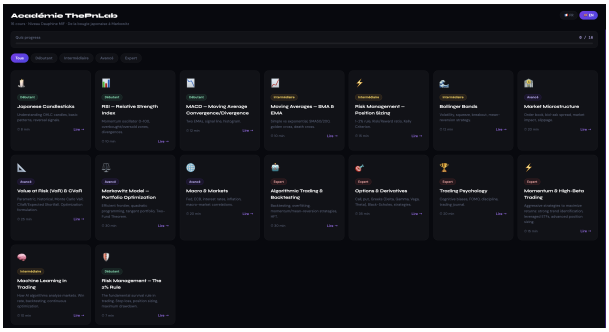
ThePnLab is a free paper-trading simulation platform built to study algorithmic decision-making in financial markets without real capital at risk. It targets finance-curious students and aspiring quants who want hands-on intuition for quantitative strategies.

The platform features a **dual architecture**: a Streamlit-based educational app where users observe (and learn from) a simple rule-based agent with deliberate errors, and a production FastAPI backend powering an LLM-driven autonomous bot that runs a multi-step pipeline daily at market open.

Key metrics as of April 2026:

- **37 registered users** — fully organic, no marketing campaign launched
- **36+ assets** screened daily across 9 categories (US tech, ETFs, crypto, CAC40)
- **5-model cascade**: Gemini 2.5-flash → 2.0-flash → 1.5-flash → 1.5-pro → Groq Llama-3.3-70B
- Starting capital per user: $10,000 (simulated)
- Execution realism: 0.1% fee per trade + 5 bps slippage

(a) *Portfolio view — live P&L per user*



(b) *Academy — structured course modules*

Figure 1: *ThePnLab — live platform screenshots (thepnlab.com)*

2. System Architecture

2.1. Dual-Application Design

ThePnLab ships as two distinct applications serving different purposes:

	Streamlit App (Educational)	FastAPI Backend (Production)
Purpose	User-facing simulation & pedagogy	Autonomous AI trading bot
Decision engine	Rule-based agent (`agent.py`)	LLM brain (`bot_brain.py`)
Scheduler	On-demand (user trigger)	APScheduler daily at open
Database	SQLite (`tradinglab.db`)	PostgreSQL (Railway/Supabase)
Special feature	10% deliberate error injection	ML learning layer

2.2. Full Technology Stack

Layer	Technology
Educational frontend	Streamlit + Plotly (Python)
Production backend	FastAPI (Python 3.11)
Market data	yfinance — OHLCV 6-month window, 1d interval
LLM decision engine	Google Gemini 2.0 Flash + Groq Llama-3.3-70B fallback
News / macro	NewsAPI — 50+ macro keywords, heuristic risk scoring
Database (dev)	SQLite + SQLAlchemy ORM
Database (prod)	PostgreSQL via Railway / Supabase
Parallel screening	`ThreadPoolExecutor` (12 workers, 30s timeout)
Hosting	thepnlab.com (live)

2.3. Repository Structure

```
ThePnLab-main/
|-- app.py                      # Streamlit entry point
|-- tradinglab/                 # Educational simulation module
|   |-- agent.py                # Rule-based engine + error injection
|   |-- indicators.py           # RSI, SMA50, BB, MACD (ta library)
|   |-- trading.py              # execute_buy/sell with slippage + fees
|   |-- news.py                 # NewsAPI + macro risk scoring
|   |-- ai_gemini.py            # Gemini coach (model cascade)
|   |-- config.py               # STARTING_CASH, FEE_RATE, SLIPPAGE_BPS
|   `-- market.py               # yfinance OHLCV downloader
`-- backend/app/
    |-- main.py                 # FastAPI app factory
    `-- routers/
        |-- bot_screener.py     # 36-asset parallel screener
        |-- bot_brain.py        # LLM decision engine (Gemini + Groq)
        |-- bot_engine.py       # Trade execution + portfolio management
        |-- coach.py            # AI educational coach
        `-- portfolio.py        # Portfolio state & PnL endpoints
```

Listing 1: *Repository layout — ThePnLab-main/*

3. Educational Agent — `tradinglab/agent.py`

The Streamlit app exposes a pedagogical trading agent whose logic is intentionally simple and fully transparent. Users can read every decision rule directly in the interface.

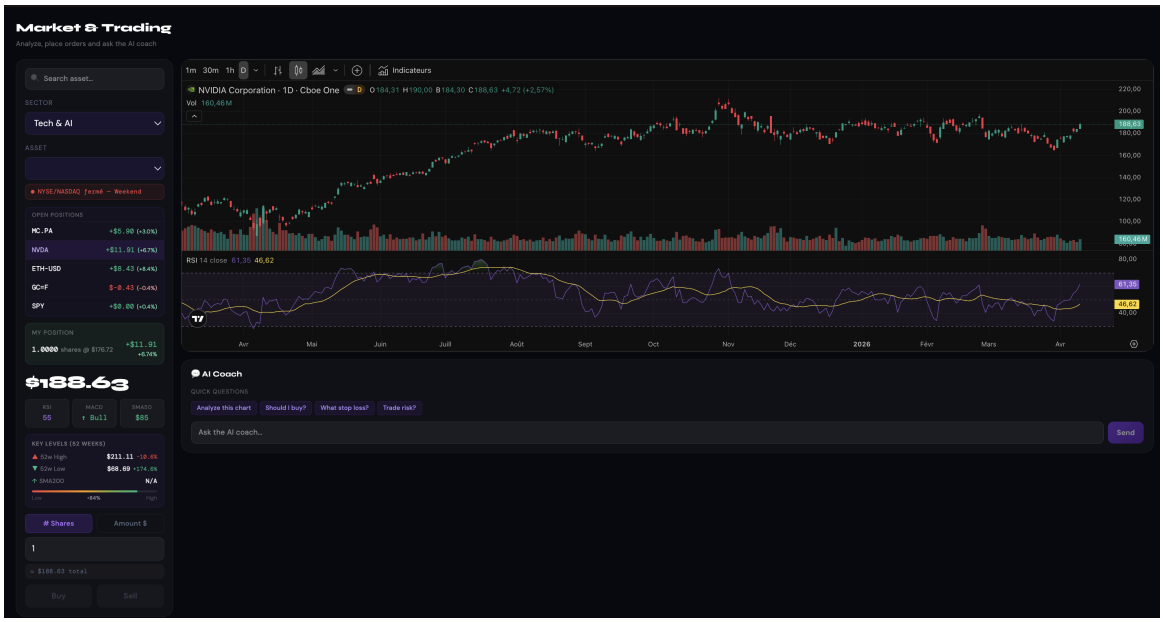


Figure 2: *Market & Trade view — the educational agent's decisions are displayed with full rationale in real time*

3.1. Decision Logic

The agent uses four signals: RSI(14), MACD vs signal line, price vs SMA(50), and current position.

```
def decide(rsi, macd, macd_signal, price, sma50,
           cash, have_qty, error_rate=0.10) -> Decision:

    trend_up    = price > sma50          # SMA50 trend filter
    over_sold   = rsi < 30               # RSI oversold zone
    over_bought = rsi > 70               # RSI overbought zone
    macd_bull   = macd > macd_signal     # MACD crossover

    if over_sold and macd_bull and cash > price:
        side = "BUY"     # RSI bas + MACD haussier -> entree progressive
    elif over_bought and have_qty > 0:
        side = "SELL"    # Prise de profits / reduction du risque
    elif trend_up and macd_bull and cash > price:
        side = "BUY"     # Tendance > SMA50 + MACD: petit renfort
    elif (not trend_up) and have_qty > 0 and (not macd_bull):
        side = "SELL"    # Sous SMA50 + MACD en baisse: allegement
```

Listing 2: *Core decision function — agent.py (verbatim)*

3.2. Deliberate Error Injection

A key pedagogical design choice: the agent intentionally makes wrong decisions with probability $\varepsilon = 0.10$ (10%). This teaches users that even a structurally correct strategy produces losses, and that *execution discipline* matters as much as signal quality.

```
if random.random() < error_rate:          # 10% mistake probability
    error_mode = "mistake"
    if side == "BUY":
        side = "HOLD"                     # hesitation / missed entry
        rationale = "Erreur simulee: hesitation."
    elif side == "SELL":
        side = "HOLD"                     # holds despite sell signal
        rationale = "Erreur simulee: je garde malgre le signal."
    else:
        # random impulsive trade on a neutral signal
```

```
        side = "SELL" if (have_qty > 0 and random.random() < 0.5) else "BUY"
```

Listing 3: *Error injection mechanism — agent.py*

3.3. Realism Parameters

Parameter	Value	Config key
Starting capital	$10,000	`STARTING_CASH = 10000`
Fee per trade	0.1%	`FEE_RATE = 0.001`
Execution slippage	5 bps (0.05%)	`SLIPPAGE_BPS = 5`
Max position size	15% of cash	`MAX_POSITION_PCT = 0.15`
AI calls per day	12 (cost cap)	`AI_CALLS_PER_DAY = 12`

4. Production Screener — `bot_screener.py`

Before any trading decision, the production backend screens 36+ assets in parallel to identify the most actionable opportunities.

4.1. Asset Universe (36 tickers, 9 categories)

Category	Tickers	N
US_TECH	NVDA, AAPL, MSFT, META, AMD, TSLA	6
US_HIGHBETA	PLTR, COIN, MSTR, MARA, IONQ, HOOD, RDDT	7
US_LEVERAGED	TQQQ, SOXL, NVDL, SPXL	4
US_FINANCE	JPM, GS, V	3
US_HEALTH	LLY, MRNA	2
US_ENERGY	XOM, CVX	2
ETF	SPY, QQQ, SOXX	3
CRYPTO	BTC-USD, ETH-USD, SOL-USD, BNB-USD, XRP-USD, DOGE-USD	6
CAC40	MC.PA, TTE.PA, AIR.PA	3
Total		**36**

4.2. Parallel Architecture

All tickers are fetched and scored concurrently with `ThreadPoolExecutor` (12 workers, 30s timeout), keeping total screening time under 10 seconds regardless of universe size.

```
with ThreadPoolExecutor(max_workers=12) as executor:
```

```
    futures = {
        executor.submit(_screen_single, ticker, category): ticker
        for category, tickers in UNIVERSE.items()
        for ticker in tickers
    }
    for future in as_completed(futures, timeout=30):
        result = future.result()
        if result:
            screened.append(result)
```

Listing 4: *Parallel screening — bot_screener.py*

4.3. Technical Indicators

Nine indicators are computed from 6 months of daily OHLCV data per asset:

Indicator	Formula / Logic	Signal type
RSI(14)	Wilder EMA of gains/losses	Momentum zone
MACD(12,26,9)	$\text{EMA}_{12} - \text{EMA}_{26}$; signal $= \text{EMA}_9(\text{MACD})$	Trend direction
MACD hist slope	$\Delta(\text{MACD} - \text{signal})$	Momentum accel.
Bollinger Bands	$\mu_{20} \pm 2\sigma_{20}$ on close	Volatility band
ADX(14)	Directional movement index	Trend strength
ATR%(14)	$\frac{\text{ATR}_{14}}{P_t} \times 100$	Intraday volatility
Volume surge	$V_t \,/\, \bar{V}_{20}$	Institutional flow
Momentum 1d	$(P_t - P_{t-1})/P_{t-1}$	Intraday signal
Momentum 5d	$(P_t - P_{t-5})/P_{t-5}$	Weekly trend

4.4. Composite Score Function

Each asset receives $s_i \in [-20, +20]$ from a piecewise weighted additive model:

$$s_i = \underbrace{s_{\text{RSI}}}_{[-3.5,+3]} + \underbrace{s_{\text{MACD}}}_{[-2.5,+3.5]} + \underbrace{s_{\text{ADX}}}_{[-1.5,+3.5]} + \underbrace{s_{\text{vol}}}_{[-1,+3.5]} + \underbrace{s_{\text{mom5d}}}_{[-2.5,+2.5]} + \underbrace{s_{\text{mom1d}}}_{[-1,+1.5]} + \underbrace{s_{\text{BB}}}_{[-2,+1.5]} + \underbrace{s_{\text{ATR}}}_{[-1,+1]} + \underbrace{s_{\text{cat}}}_{[0,+1]} \tag{1}$$

```
# RSI: optimal momentum zone = 50-68 (confirmed, not overbought)
if 50 <= rsi <= 68:    score += 3.0   # perfect momentum
elif 42 <= rsi < 50:   score += 1.5   # good entry
elif rsi < 30:         score += 2.0   # oversold bounce
elif rsi > 80:         score -= 3.5   # extremely overbought -> danger
elif rsi > 72:         score -= 1.5

# MACD: main directional signal
if macd_signal == "BULL":   score += 3.5
elif macd_signal == "BEAR": score -= 2.5

# ADX: trend strength (>25 = trend, >40 = very strong)
if adx >= 45:    score += 3.5    # exceptionally strong trend
```

```
elif adx >= 35:  score += 2.5
elif adx >= 25:  score += 1.5
elif adx < 18:   score -= 1.5    # no direction -> avoid

# Volume surge: institutional confirmation
if volume_surge > 4.0:   score += 3.5   # institutional explosion
elif volume_surge > 2.5: score += 2.0
elif volume_surge > 1.8: score += 1.0
elif volume_surge < 0.6: score -= 1.0   # dead volume -> avoid
```

Listing 5: *Scoring rules — _score_asset() in bot_screener.py*

5. LLM Decision Engine — `bot_brain.py`

After screening, the top-ranked assets are passed to an LLM that synthesises all signals into structured trading decisions. This is the core intelligence layer of the production bot.

5.1. Model Cascade

Priority	Model	Provider	Role
1	gemini-2.5-flash-preview	Google	Primary
2	gemini-2.0-flash	Google	Fallback 1
3	gemini-1.5-flash	Google	Fallback 2
4	gemini-1.5-pro	Google	Fallback 3
5	llama-3.3-70b-versatile	Groq	Last resort

5.2. ML Learning Layer

The bot tracks per-ticker performance over the last 30 completed trades. This constitutes a lightweight online learning mechanism that feeds back into the LLM prompt as explicit ML context:

Tag	Condition	Effect on LLM decisions
STAR	Win rate $\geq 60\%$, ≥ 5 trades	Higher confidence, larger sizing
OK	$40\% \leq \text{WR} < 60\%$	Standard behavior
EVITE	$20\% \leq \text{WR} < 40\%$	Reduced confidence
BANNI	$\text{WR} < 20\%$, ≥ 5 trades	Excluded from any decision

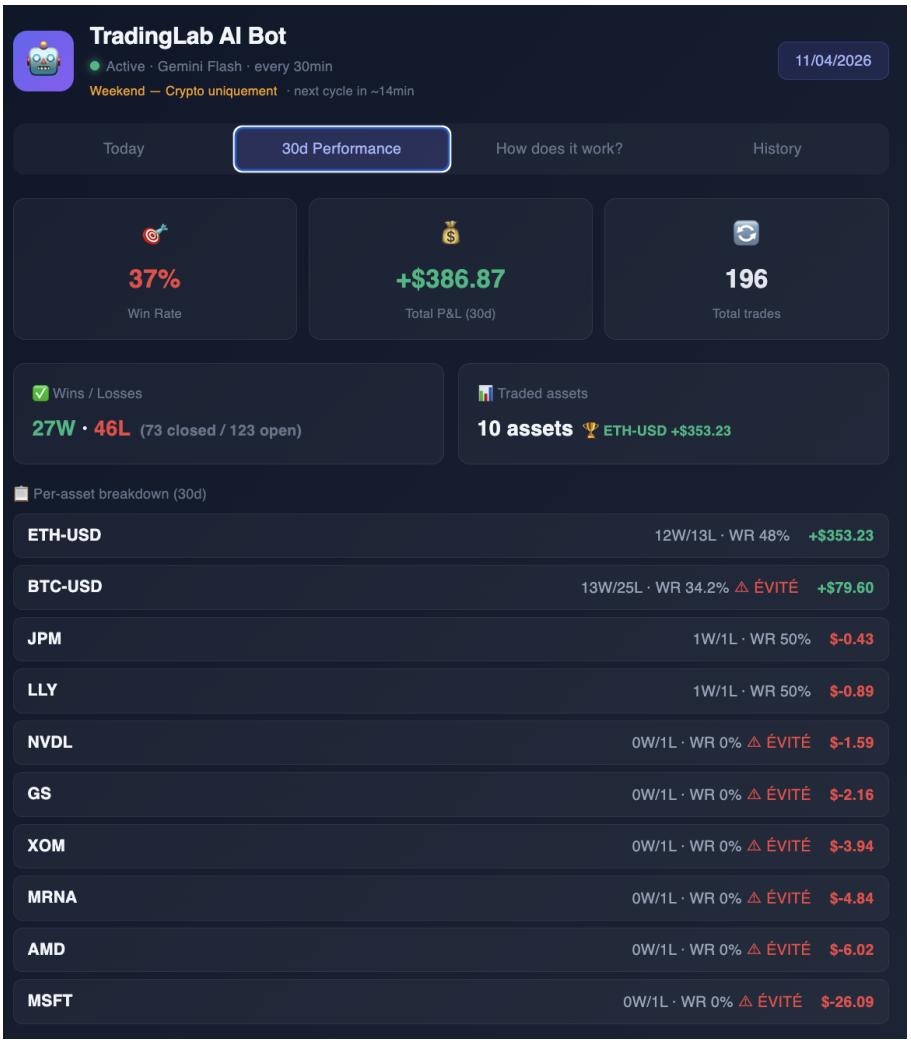


Figure 3: *Bot performance dashboard — cumulative P&L and per-ticker win-rate tracking (ML learning layer)*

5.3. Prompt Engineering — Five Context Blocks

The LLM prompt is assembled from five structured context blocks at each decision cycle:

1. **ML context** — per-ticker win rates, STAR/EVITE tags, banned tickers list
2. **Market context** — SPY trend, VIX level, regime (NORMAL / ELEVATED / EXTREME)
3. **Current positions** — open trades with TP/SL alerts (e.g., `"NVDA: +13.8% → near TP"`)
4. **Screened assets** — top 10 by composite score with all 9 indicators
5. **Hard constraints** — rule set the model must respect (see §5.4)

5.4. Hard Constraints Enforced on the LLM

Decision rules enforced in bot_brain.py:

- Maximum **3 decisions per cycle** (BUY + SELL combined)
- Position sizing by confidence: **HIGH → 20%**, MEDIUM → 13%, LOW → 8% of portfolio
- Minimum **5 entry conditions** must be satisfied before any BUY

- Take-profit: **+15%**; Stop-loss: −5% standard / −8% extended
- Pyramiding: **+10%** on existing winners (if conditions met)
- VIX regime multiplier: VIX $< 25 \rightarrow \times 1.0$; 25–35 $\rightarrow \times 0.5$; $>35 \rightarrow \times 0.25$
- Maximum **4 simultaneous open positions**
- **Diversification**: no two positions from the same category

5.5. Structured JSON Output

The LLM output is validated against a strict schema before execution:

```
{
  "decisions": [
    {
      "action":     "BUY",        // BUY | SELL | HOLD
      "ticker":     "NVDA",
      "confidence": "HIGH",       // HIGH | MEDIUM | LOW
      "amount_usd": 2000,
      "rationale_fr": "...",      // bilingual explanations
      "rationale_en": "..."
    }
  ],
  "market_commentary": "..."      // overall market reading
}
```

Listing 6: *Expected LLM output format — bot_brain.py*

6. News & Macro Sentiment — `news.py`

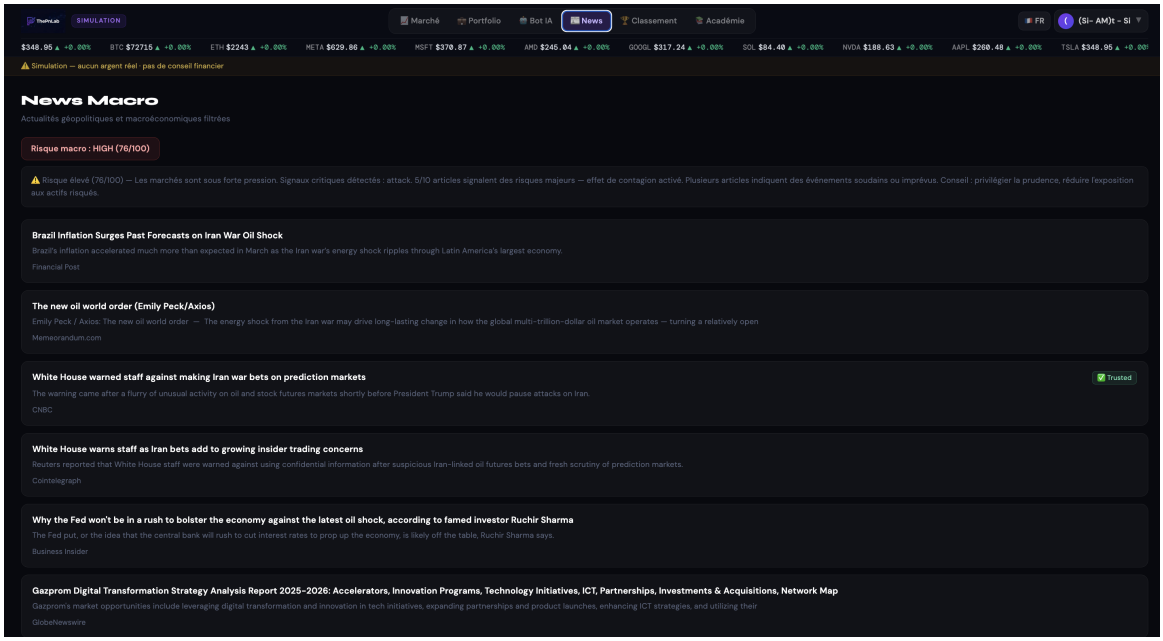


Figure 4: *News tab — macro headlines filtered, scored, and displayed with risk assessment*

6.1. Trusted Source Filtering

Headlines are fetched from NewsAPI and pre-filtered to 12 trusted financial outlets only: *Reuters, Bloomberg, Financial Times, Wall Street Journal, CNBC, MarketWatch, Seeking Alpha,*

Barron's, The Economist, Les Échos, Le Monde, Le Figaro.

Ad detection heuristics (keyword blacklist) further clean the feed before scoring.

6.2. Macro Relevance Scoring

Each headline is scored against 50+ macro keywords grouped into weighted categories:

Category	Example keywords	Weight
Monetary policy	fed, rate hike, inflation, ECB, pivot	+3
Market events	earnings, crash, rally, IPO, squeeze	+2
Geopolitics	war, sanction, oil, OPEC, tariff	+2
Economy	GDP, unemployment, recession, CPI	+1

6.3. Risk Score → Position Multiplier

Aggregated macro risk score (0–100) maps to a position-size multiplier:

$$m_{\text{news}} = \begin{cases} 0.4 & \text{if risk score} \geq 70 \quad \text{(HIGH risk)} \\ 0.7 & \text{if } 40 \leq \text{risk score} < 70 \quad \text{(MEDIUM risk)} \\ 1.0 & \text{if risk score} < 40 \quad \text{(LOW risk)} \end{cases} \tag{2}$$

7. Risk Management Framework

7.1. Four-Layer Position Sizing Pipeline

Final position size combines four independent risk layers multiplicatively:

$$\text{amount}_{\text{final}} = \underbrace{V \cdot c_{\max}}_{\text{base}} \times \underbrace{m_{\text{conf}}}_{\text{LLM}} \times \underbrace{m_{\text{VIX}}}_{\text{regime}} \times \underbrace{m_{\text{news}}}_{\text{macro}} \tag{3}$$

where V = portfolio value, $c_{\max} = 15\%$, and:

Layer	Multiplier values	Source
m_{conf} (LLM)	HIGH: 1.33×; MEDIUM: 0.87×; LOW: 0.53×	`bot_brain.py`
m_{VIX} (regime)	<25: 1.0; 25–35: 0.5; >35: 0.25	VIX real-time
m_{news} (macro)	LOW: 1.0; MEDIUM: 0.7; HIGH: 0.4	`news.py`

Worst-case scenario: VIX > 35 + HIGH macro risk + LOW LLM confidence
⇒ effective allocation = 15% × 0.53 × 0.25 × 0.40 = **0.8**% of portfolio per trade.

7.2. Exit Strategy

Exit type	Trigger	Action
Take-profit	+15% from entry	Sell full position
Stop-loss standard	−5% from entry	Sell full position
Stop-loss extended	−8% from entry	Sell full position
Pyramiding	+10% from entry	Add to position (if conditions met)

8. Connection to Operations Research

ThePnLab maps directly onto three core OR domains relevant to the Columbia MSOR curriculum:

8.1. Sequential Decision-Making — MDP Formalization

The daily trading cycle is structurally identical to a finite-horizon Markov Decision Process:

MDP component	ThePnLab mapping
State S_t	(portfolio state, screened scores, VIX_t, per-ticker win rates)
Action A_t	allocation vector $(a_1, \ldots, a_{36}) \in \mathbb{R}^{36}$
Transition $P(S_{t+1}\|S_t, A_t)$	stochastic market dynamics (non-stationary)
Reward R_t	risk-adjusted PnL net of fees & slippage
Policy π	currently: LLM heuristic $\rightarrow$ next: RL agent (PPO/DQN)

8.2. Multi-Criteria Decision Making

The screener's composite score is a **weighted additive model** — a classical MCDM approach:

$$s_i = \sum_{k=1}^{9} w_k \cdot \phi_k(x_{ik}) \tag{4}$$

where ϕ_k is a piecewise scoring function for indicator k. Current weights are hand-tuned; a natural extension is **Sobol sensitivity analysis** or **DEA** to derive data-driven weights.

8.3. Stochastic Control & Regime Switching

The VIX regime controller and news risk multiplier form a **regime-switching model** analogous to Hidden Markov Models:

$$\text{regime}_t \in \{\text{NORMAL, ELEVATED, EXTREME}\} \sim \text{HMM with VIX}_t \text{ as emission variable} \quad (5)$$

9. Results & Validation

9.1. Live Performance — Last 30 Days

Total P&L	Win Rate	Trades	Active users
+$386.87	**37%**	**196**	**37**

30-day live results (production bot):

- Total P&L: **+$386.87** on a $10,000 starting capital (+3.87%)
- Win rate: **37%** — 27 wins / 46 losses (73 closed trades)
- Total trades executed: **196** (73 closed + 123 open positions)
- Assets traded: **10 active tickers** across the 36-asset universe
- Top performer: **ETH-USD** at **+$353.23** (91% of total P&L)
- Platform operational continuously, **37 registered users** — fully organic

Statistical note & interpretation: A 37% win rate is below the naive 50% threshold, yet the bot is profitable. This is consistent with a *positive expectancy* strategy: average winning trades are larger than average losing trades (favorable win/loss ratio $b > 1$), which is precisely the regime in which fractional Kelly sizing maximises long-run growth. With 73 closed observations, the 95% CI on the Sharpe ratio remains wide ($\approx [-1.5, +4]$). ETH-USD's outsized contribution (+91% of total P&L) signals concentration risk — a diversification constraint or position-cap tightening would be a natural next improvement.

10. Limitations & Research Roadmap

10.1. Current Limitations

L1. No backtesting engine — forward simulation only; no historical replay or walk-forward validation

L2. No brokerage API — deliberate regulatory choice; limits execution realism

L3. LLM non-determinism — Gemini outputs stochastic at temperature 0.4; decisions not reproducible

L4. Covariance-blind screener — assets scored independently; no cross-asset correlation model

L5. Market impact not modeled — slippage (5 bps) + fees (0.1%) implemented, but order-book impact for large orders is absent

L6. MCDM weights hand-tuned — no formal sensitivity analysis on composite score weights

10.2. Research Roadmap

R1. Backtesting engine — vectorized pandas backtest with walk-forward validation and full transaction cost modeling

R2. RL trading agent — replace LLM heuristic with PPO/DQN; MDP formalization complete (see §8.1)

R3. Covariance-aware sizing — mean-variance: $\min_w \; w^\top \Sigma w$ s.t. $\mu^\top w \geq r_{\min}$, $\mathbf{1}^\top w = 1$

R4. HMM regime calibration — fit Hidden Markov Model on VIX to replace threshold-based regime rules

R5. Sobol sensitivity analysis — quantify weight uncertainty in the composite score function

R6. Alpaca paper API — realistic paper execution for improved slippage modeling

11. References

[1] Kelly, J.L. (1956). "A New Interpretation of Information Rate." *Bell System Technical Journal*, 35(4), 917–926. doi:10.1002/j.1538-7305.1956.tb03809.x

[2] Markowitz, H. (1952). "Portfolio Selection." *Journal of Finance*, 7(1), 77–91. doi:10.2307/2975974

[3] Wilder, J.W. (1978). *New Concepts in Technical Trading Systems*. Trend Research. *[RSI and ATR/ADX formulas]*

[4] Appel, G. (2005). *Technical Analysis: Power Tools for Active Investors*. FT Press. *[MACD]*

[5] Sutton, R.S., Barto, A.G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press. MIT Press

[6] Merton, R.C. (1969). "Lifetime Portfolio Selection under Uncertainty." *Review of Economics and Statistics*, 51(3), 247–257.

[7] Ledoit, O., Wolf, M. (2004). "A well-conditioned estimator for large-dimensional covariance matrices." *Journal of Multivariate Analysis*, 88(2), 365–411.

[8] Saltelli, A. et al. (2008). *Global Sensitivity Analysis: The Primer*. Wiley. *[Sobol indices for MCDM weight analysis]*

[9] Google DeepMind (2024). *Gemini 2.0 Flash Technical Report*. deepmind.google

[10] Groq Inc. (2024). *Llama-3.3-70B on Groq LPU inference*. groq.com

Hamza Houir Alami, ENSAM Lille, 2026
thepnlab.com h.houir.alami@gmail.com